Python 3.12 AsyncIO & High-Performance Coroutines

Presented By sam

Agenda

- • Introduction to AsyncIO
- • Understanding Coroutines
- • Writing Simple Async Code

Introduction to AsyncIO

- • AsyncIO for non-blocking I/O
- • Example: Reading a file

```
// PYTHON 3.12.8
# Concept: Introduction to AsyncIO
message = "Introduction to AsyncIO Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Reading a File

```
            // PYTHON 3.12.8
async def read_file(file_path):
    with open(file_path, 'r') as file:
        async for line in file:
            print(line)
    return
read_file('example.txt')
```

Understanding Coroutines

- • Coroutines are lightweight threads
- • Example: Simple coroutine

```
// PYTHON 3.12.8
# Concept: Understanding Coroutines
message = "Understanding Coroutines Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

Simple Coroutine

```
            // PYTHON 3.12.8
async def simple_coroutine(num):
    for i in range(num):
        print(f'Coroutine {num} - {i}')
        await asyncio.sleep(1)
asyncio.run(simple_coroutine(3))
```

Writing Simple Async Code

- • Combining coroutines with AsyncIO
- • Example: Async function with coroutine

```
// PYTHON 3.12.8
# Concept: Writing Simple Async Code
message = "Writing Simple Async Code Executed in Python 3...
print(message)
return message
asyncio.run(demonstrate_concept())
```

Async Function with Coroutine

```
            // PYTHON 3.12.8
async def async_function(num):
    for i in range(num):
        await simple_coroutine(i)
asyncio.run(async_function(3))
```

Exercise: Async Function with Coroutine

```
// PYTHON 3.12.8
async def exercise_function(num):
    for i in range(num):
        await simple_coroutine(i)
asyncio.run(exercise_function(3))
```

Summary

- • AsyncIO for non-blocking I/O
- • Understanding coroutines
- • Writing simple async code

Closing

- Thank You
- Q&A
- References